

Memoria *cache*: operații de scriere si algoritmi de înlocuire

7. Operațiile de scriere în *cache*

Pornind de la rata de hit (***h***) am dedus ecuația care redă timpul mediu de acces la memorie:

$$t_a = t_c + (1-h) \cdot t_m$$

Ecuația a fost dedusă evaluând funcționarea memoriei *cache* strict pe operațiile de citire din memorie.

Operațiile de scriere vor adăuga un timp adițional în ecuația de mai sus, timp care depinde de mecanismul utilizat pentru menținerea consistenței datelor.

Există două mecanisme alternative de actualizare a conținutului MP în cadrul operațiilor de scriere:

- mecanismul ***write-through***
- mecanismul ***write-back***

7. Operațiile de scriere

Mecanismul *write-through*:

- În mecanismul *write-through*, orice operație de scriere în *cache* este repetată și în MP, în același timp, asigurând astfel consistența datelor.

- Timpul mediu de acces:

$$t_a = t_c + (1-h) \cdot t_l + w \cdot (t_m - t_c)$$

t_l - timpul de transfer a unei linii în *cache*

w - procentajul de scrieri în memorie

($t_l = t_m$ – bus extins $t_l = l \cdot t_m$ – bus simplu comutat de la modul la modul)

- Timpul mediu de acces în cazul strategiei *no fetch on write*:

$$t_a = t_c + (1-w) \cdot (1-h) \cdot t_l + w \cdot (t_m - t_c) = (1-w) \cdot [t_c + (1-h) \cdot t_l] + w \cdot t_m$$

- Relația poate fi dedusă și prin luarea în considerare a tuturor combinațiilor posibile (*read/write* cu *hit/miss*):

$$t_a = \underbrace{(1-w) \cdot h \cdot t_c}_{\text{Read-hit}} + \underbrace{(1-w) \cdot (1-h) \cdot (t_c + t_l)}_{\text{Read-miss}} + \underbrace{w \cdot h \cdot t_m}_{\text{Write-hit}} + \underbrace{w \cdot (1-h) \cdot t_m}_{\text{Write-miss}}$$

7. Operațiile de scriere

Mecanismul *write-back*:

- În mecanismul *write-back*, scrierea unei linii în MP se amână până în momentul înlocuirii liniei respective în *cache*:

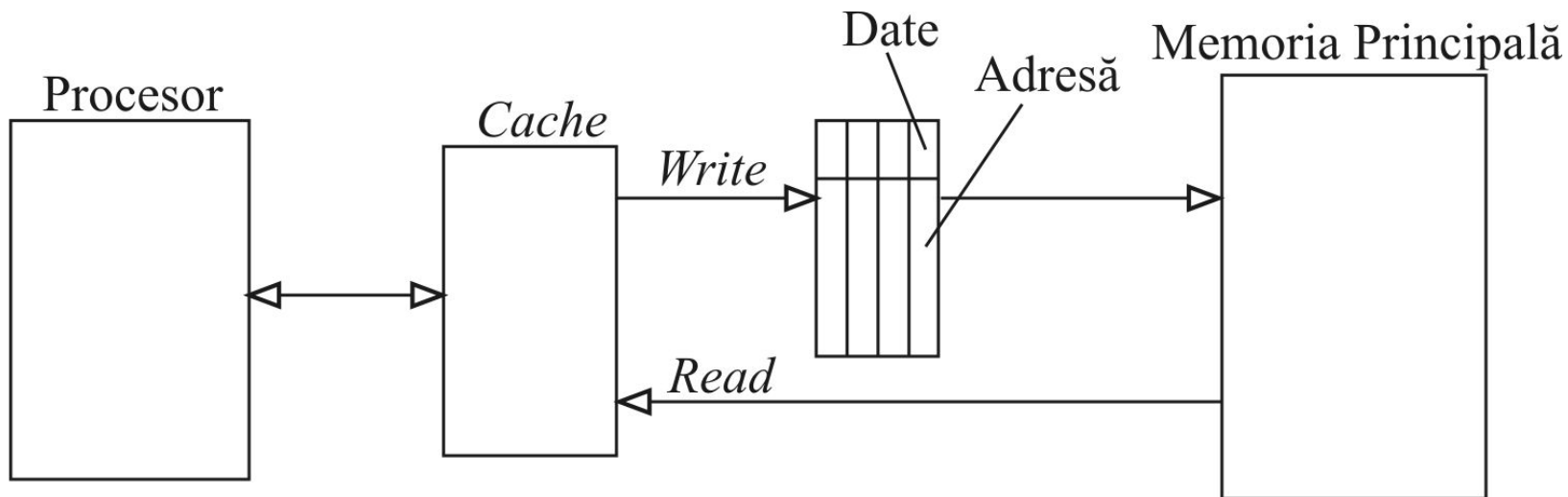
$$t_a = t_c + (1-h) \cdot t_l + (1-h) \cdot t_l = t_c + 2 \cdot (1-h) \cdot t_l$$

- Un mecanism *write-back* îmbunătățit scrie înapoi în MP doar liniile care au fost alterate (modificate) în *cache*:

$$t_a = t_c + (1-h) \cdot t_l + w_l \cdot (1-h) \cdot t_l = t_c + (1-h) \cdot (1+w_l) \cdot t_l$$

unde w_l reprezintă probabilitatea ca o linie *cache* să fie alterată (procentul de linii alterate).

7. Operațiile de scriere



Cache cu *buffer* de scriere (*write-buffer*)

8. Algoritmi de înlocuire

Algoritmii de înlocuire sunt de 2 tipuri:

- algoritmi bazați pe gradul de utilizare a liniilor *cache*
- algoritmi care nu se bazează pe gradul de utilizare a liniilor *cache*

Algoritmii utilizați sunt:

- **Algoritmul de înlocuire aleatoare**
- **Algoritmul *first-in first-out* (FIFO)**
- **Algoritmul *least recently used* (LRU)**

Variante de implementare a algoritmului LRU în *hardware*:

- numărătoare (contoare)
- stivă de registre
- matrici de referințe
- metode aproximative

8. LRU implementat cu contoare

Varianta 1:

Contoarele sunt incrementate periodic (la intervale regulate) și când se referă o linie contorul asociat acesteia este resetat.

Deci valoarea fiecărui contor indică "vârsta" liniei, contorizată de la ultima referință la linia respectivă. În momentul înlocuirii va fi înlocuită linia cu vârsta cea mai mare.

Dezavantaj: **saturarea contoarelor !**

Varianta 2:

Acces cu *hit*: contorul asociat liniei de *hit* este resetat la 0, indicând că linia este cea mai recent referită, și toate contoarele care conțin o valoare mai mică decât valoarea originală (dinaintea resetării) a contorului liniei de *hit* sunt incrementate cu 1; toate contoarele care conțin o valoare mai mare rămân neafectate.

Acces cu *miss* când *cache*-ul nu este plin: contorul asociat liniei care se încarcă în *cache* este resetat la 0 și toate celelalte contoare sunt incrementate cu 1.

Acces cu *miss* când *cache*-ul este plin: linia care are contorul pe valoarea maximă este selectată pentru înlocuire, înlocuită, contorul asociat resetat la 0 și toate celelalte contoare incrementate cu 1.

Avantaj: contorizând vârste relative, **previne saturarea** contoarelor !

8. LRU implementat cu contoare

Exemplu: algoritmul LRU implementat cu contoare pe un set asociativ cu 4 linii.

Locații referite		Stare contoare				Acțiuni în <i>cache</i>
<i>Tag-ul din adresă</i>	<i>Hit/Miss</i>	C_0	C_1	C_2	C_3	
–	Inițializare	0	0	0	0	–
5	<i>Miss</i>	0	1	1	1	Linia 0 încărcată
6	<i>Miss</i>	1	0	2	2	Linia 1 încărcată
5	<i>Hit</i>	0	1	2	2	Linia 0 accesată
7	<i>Miss</i>	1	2	0	3	Linia 2 încărcată
8	<i>Miss</i>	2	3	1	0	Linia 3 încărcată
6	<i>Hit</i>	3	0	2	1	Linia 1 accesată
9	<i>Miss</i>	0	1	3	2	Linia 0 înlocuită
10	<i>Miss</i>	1	2	0	3	Linia 2 înlocuită

Algoritmul LRU implementat cu contoare pe un set asociativ cu 4 linii.

8. LRU implementat cu stivă de registre și matrice de referințe

Exemplu: LRU implementat cu stivă de registre și matrice de referințe pe un set asociativ cu 4 linii.

Stiva de registre:

Când se accesează linia i ($0 \leq i \leq 2n-1$), începând cu registrul din vârful de sus al stivei, toate valorile din registre sunt deplasate cu o poziție în jos până la registrul care conține valoarea i (egală cu identificatorul liniei accesate). Următoarele registre rămân neafectate de deplasare, iar registrul din vârful superior al stivei va fi încărcat cu valoarea i (identificatorul liniei accesate).

Matricea de referințe:

Când linia i este referită:

- toți biții din rândul i al matricii triunghiulare sunt setați la 1.
- toți biții din coloana i sunt resetați la 0.

Linia cea mai puțin recent referită este cea care:

- pe rand are numai biți de 0
- Pe coloana are numai biți de 1

8. LRU implementat cu stivă de registre și matrice de referințe

Exemplu: LRU implementat cu **stivă de registre** și respectiv **matrice de referințe** pe un set asociativ cu 4 linii.

1

2

R0	2
R1	
R2	
R3	

2

1

	1
	2

3

3

	3
	1
	2

4

0

	0
	3
	1
	2

2

5

3

	3
	0
	1
	2

2

6

2

	2
	3
	0
	1

1

7

1

	1
	2
	3
	0

0

Secvența de accese
în setul asociativ

Linii *cache* referite

Starea stivei de
registre după
fiecare referință

Linia cea mai puțin
recent utilizată

a) Metoda stivei de registre

2

3			0
2	1	1	
1			
0			

0 1 2 3

1

3		0	0
2	1	0	
1	1		
0			

0 1 2 3

3

3	1	1	1
2	1	0	
1	1		
0			

0 1 2 3

0

3	0	1	1
2	0	0	
1	0		
0			

0 1 2 3

3

3	1	1	1
2	0	0	
1	0		
0			

0 1 2 3

2

3	1	1	0
2	1	1	
1	0		
0			

0 1 2 3

1

3	1	0	0
2	1	0	
1	1		
0			

0 1 2 3

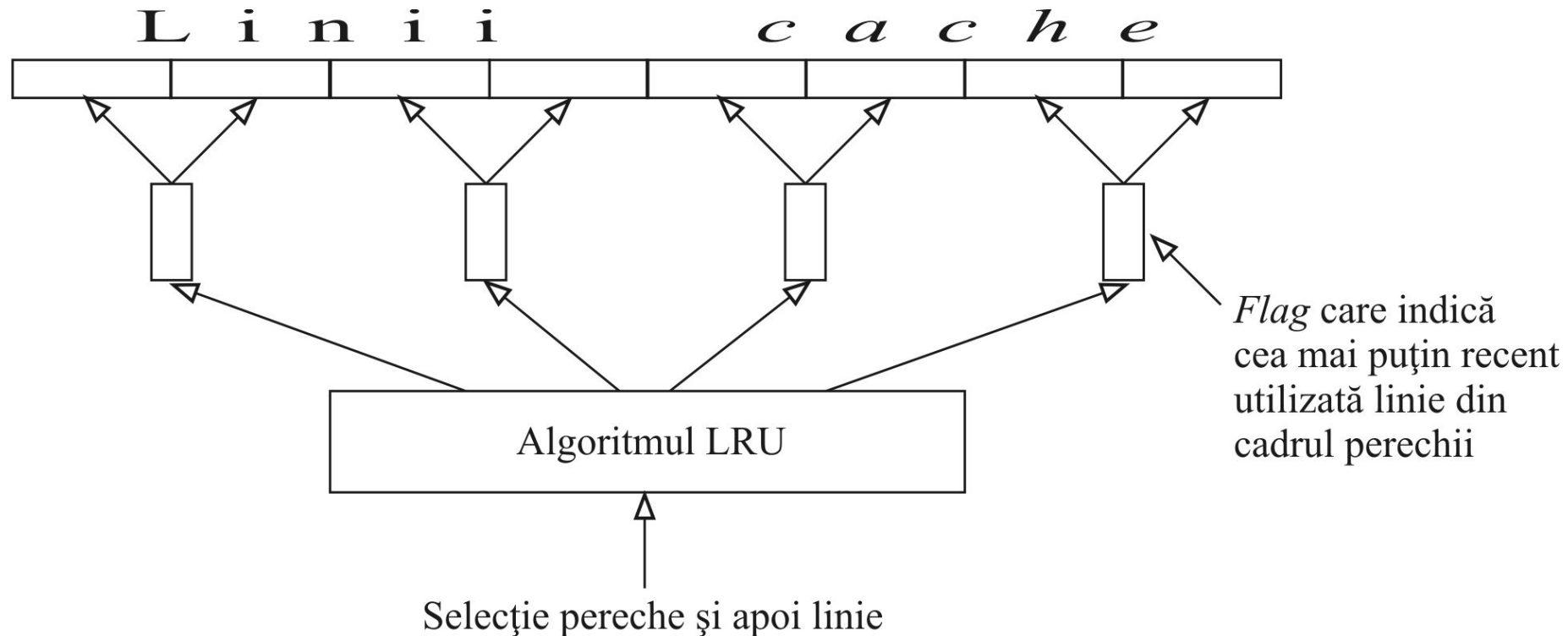
Linii *cache* referite

Matricea de stare
după fiecare referință

Linia cea mai puțin
recent utilizată

b) Metoda matricii de referințe

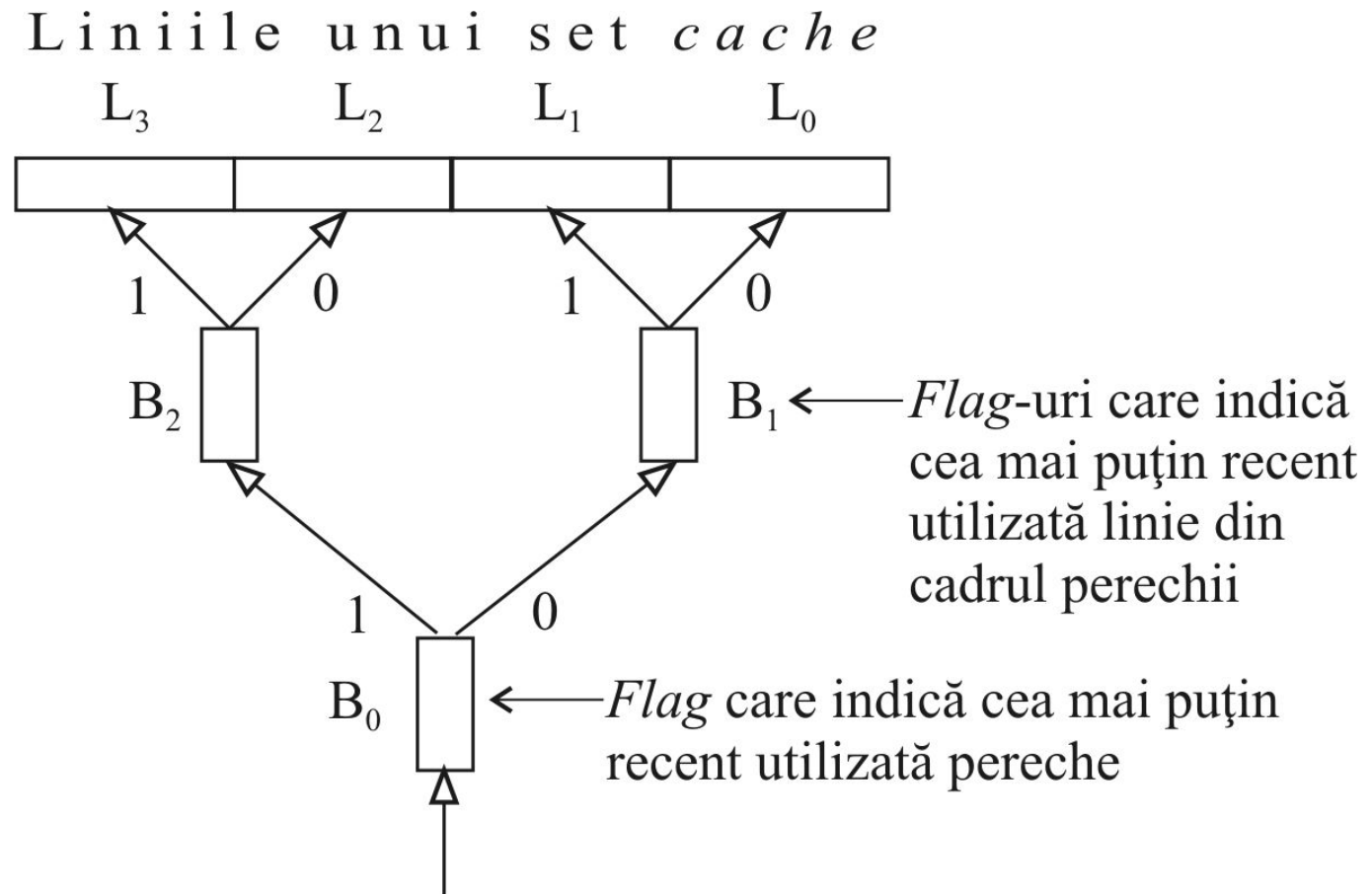
8. LRU – metode aproximative



Algoritm de înlocuire implementat pe doua nivele:

1. matrice de referințe
2. *flag*-uri pentru selecția celei mai recente (cele mai puțin recente) linii din cadrul perechii

8. LRU – metode aproximative



Algoritm de înlocuire utilizând selecția arborescentă implementat la Intel 486